
name: AI Semantic Hints Debug

overview: Add a debug-only GPT-4o vision analysis step that renders the PDF floorplan to an image, sends it to OpenAI, and returns structured semantic hints (page type, unit outline, space classifications, open-plan envelopes) as a debug artifact on the existing geometry-first payload.

todos:

- id: create-hints-module

- content: Create 'lib/geometryFirst/floorplanAiSemanticHints.ts' with PDF-to-image rendering, GPT-4o vision call, and JSON parsing

- status: pending

- id: add-types

- content: Add 'FloorplanAiSemanticHintsDebug' interface and field to 'GeometryFirstDebugPayload' in types.ts

- status: pending

- id: wire-route

- content: Wire 'includeAiSemanticHints' option into the API route and attach result to debug payload

- status: pending

- id: add-exports

- content: Export new function and type from index.ts

- status: pending

- id: test-run

- content: Run a test with the tower_6_B.pdf to verify end-to-end

- status: pending

isProject: false

AI Floorplan Semantic Hints (Debug-Only)

Architecture

The AI step runs **outside** the synchronous 'buildGeometryFirstDebugPayload' ? it is called from the API route where 'pdfBytes' is available, and its result is attached to the debug payload afterward. This avoids making the main debug builder async.

“mermaid

sequenceDiagram

participant Route as route.ts

participant Extract as runGeometryFirstExtraction

participant Debug as buildGeometryFirstDebugPayload

participant AI as buildFloorplanAiSemanticHints

participant OpenAI as GPT-4o Vision API

Route->>Extract: pdfBytes

```

Extract-->>Route: result (primitives)
Route->>Debug: result.primitives, opts
Debug-->>Route: debugPayload (sync)
Route->>AI: pdfBytes, opts
AI->>AI: render PDF page 1 to PNG (pdfjs-dist + @napi-rs/canvas)
AI->>AI: downscale via sharp if needed
AI->>OpenAI: base64 PNG + structured prompt
OpenAI-->>AI: strict JSON
AI-->>Route: FloorplanAiSemanticHintsDebug
Route->>Route: attach to debugPayload.aiSemanticHints

```

Files to Create / Modify

1. NEW: 'lib/geometryFirst/floorplanAiSemanticHints.ts'

Core async function with three responsibilities:

```

? PDF-to-image: Use 'pdfjs-dist' (already in deps) + '@napi-rs/canvas' (already in deps)
to render page 1 to a canvas, then export as PNG buffer. Reuse 'ensurePdfJsCanvasGlobals()'
from '[lib/geometryFirst/pdfJsNodePolyfills.ts](lib/geometryFirst/pdfJsNodePolyfills.ts)'.
Render at 2x scale (~1200-1600px long edge for typical A4/Letter). Downscale with
'sharp' if needed (cap at 2048px long edge).
? Vision API call: 'fetch("https://api.openai.com/v1/chat/completions")' with 'gpt-4o',
'response_format: { type: "json_object" }', and 'temperature: 0.15'. Same pattern as
'[app/api/analyze-floorplan/route.ts](app/api/analyze-floorplan/route.ts)' lines 301-311.
Gracefully return 'null' if 'OPENAI_API_KEY' is missing or the call fails.
? Prompt: Instruct the model to return strict JSON with coordinates in PDF page pixel
units (matching the geometry pipeline's coordinate system, not meters). Explicitly
distinguish 'enclosed_room' vs 'open_plan_subzone' for major spaces, allow KITCHEN +
LIVING/DINING to share an open-plan envelope.

```

Signature:

```

""typescript
export async function buildFloorplanAiSemanticHints(
  pdfBytes: Uint8Array,
  opts?: { renderScale?: number; maxEdgePx?: number; pageIndex?: number }
): Promise<FloorplanAiSemanticHintsDebug | null>

```

2. MODIFY: 'lib/geometryFirst/types.ts'

Add the output interface (before 'GeometryFirstDebugPayload'):

```
""typescript
export interface FloorplanAiSemanticHintsDebug {
  pageType: string
  unitOutlineHint: {
    vertices: Array<{ x: number; y: number }>
    confidence: number
    notes?: string
  }
  majorSpaces: Array<{
    label: string
    spaceType: "enclosed_room" | "open_plan_subzone"
    approxPolygon: Array<{ x: number; y: number }>
    confidence: number
  }>
  sharedOpenPlanEnvelopes: Array<{
    memberLabels: string[]
    approxPolygon: Array<{ x: number; y: number }>
    confidence: number
  }>
  auxiliarySpaces: Array<{
    label: string
    approxPolygon: Array<{ x: number; y: number }>
    confidence: number
  }>
  notes: string[]
  modelUsed: string
  renderPx: { width: number; height: number }
  latencyMs: number
  rawModelResponse?: string
}
""
```

Add to 'GeometryFirstDebugPayload':

```
""typescript
aiSemanticHints?: FloorplanAiSemanticHintsDebug
""
```

3. MODIFY: 'app/api/floorplan-geometry-first/route.ts'

? Add 'includeAiSemanticHints?: boolean' to the 'debug' options type.

? After 'buildGeometryFirstDebugPayload' returns, if 'includeAiSemanticHints' is true and

'pdfBytes' is available, call 'buildFloorplanAiSemanticHints(pdfBytes)'.
? Attach result to 'result._debugGeometryFirst.aiSemanticHints'.
? Increase 'maxDuration' from 60 to 120 (vision API can take 15-30s).

4. MODIFY: 'lib/geometryFirst/index.ts'

? Export 'buildFloorplanAiSemanticHints' and 'FloorplanAiSemanticHintsDebug' type.

Prompt Design (Key Points)

? All coordinates in PDF page pixel units (origin top-left, x right, y down)
? 'page_type': one of "residential_brochure_floorplan", "architectural_drawing",
"site_plan", "other"
? 'unit_outline_hint': approximate outer boundary vertices (clockwise loop)
? 'major_spaces[]': each with 'label', 'space_type' ('enclosed_room' | 'open_plan_subzone'),
'approx_polygon', 'confidence'
? 'shared_open_plan_envelopes[]': groups like KITCHEN + LIVING/DINING sharing one open
envelope
? 'auxiliary_spaces[]': corridors, storage, utility, balcony
? 'notes[]': free-text observations

Key Design Decisions

? **Coordinates in PDF pixels, not meters** ? matches the geometry pipeline; avoids scale
estimation errors from the model
? **Async, outside the sync debug builder** ? cleanest integration, no refactor of existing
code
? **Graceful null return** ? no API key or API failure produces 'null', pipeline continues
unaffected
? **Debug-only** ? gated behind 'includeAiSemanticHints' flag, production unchanged
? **'rawModelResponse' field** ? stores the raw JSON string for debugging prompt issues